

EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

Program Studi Teknik Instrumentasi
Semester Genap 2025/2026

PROJECT BASED LEARNING (PBL)

Sistem Pengukuran dan Kontrol Level Tangki Industri
dengan Valve
Berdasarkan Rust

Dosen Pengampu : Ahmad Radhy, S.Si., M.Si.
Durasi Pengerjaan : 1 Pekan
Bahasa Pemrograman : Rust
Metode Penilaian : Project + Ujian Lisan

Kelompok : 17

Mahasiswa 1 : Benedycta Eka Putra
NRP : 2042251055

Mahasiswa 2 : Bagas Andika Firmansyah
NRP : 2042251057

Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember

Daftar Isi

1	Pendahuluan	2
2	Studi Kasus Instrumentasi Sistem Monitoring Level Tangki	2
3	Algoritma Program	4
3.1	Flowchart Sistem	5
3.2	Penjelasan Flowchart	6
4	KONSEP PEMROGRAMAN BERBASIS OBJEK (OOP)	8
4.1	Pengertian Object Oriented Programming (OOP)	8
4.2	Penerapan OOP pada Sistem Monitoring Level Tangki	8
4.3	Konsep Encapsulation	8
4.4	Konsep Constructor	9
4.5	Keuntungan Penggunaan OOP pada Project	9
5	Implementasi Komputasi Numerik	10
6	Penjelasan Program Rust	11
6.1	Latar Belakang	11
6.2	Kode Sumber (Source Code)	11
6.3	Penjelasan Struktur Program	14
6.3.1	Struct TankMonitoring	14
6.3.2	Implementasi Method (impl)	15
6.3.3	Fungsi Helper input_f32	15
6.4	Logika Operasional Sistem	15
7	Hasil Pengujian	16
8	Analisis Hasil	16
9	Kesimpulan	17

1 Pendahuluan

Perkembangan teknologi otomasi industri saat ini semakin meningkat, terutama pada sistem monitoring dan pengendalian proses di berbagai sektor industri seperti pengolahan air, minyak dan gas, makanan dan minuman, serta manufaktur. Salah satu sistem yang sangat penting dalam dunia industri adalah sistem monitoring level tangki. Sistem ini digunakan untuk memantau ketinggian fluida di dalam tangki agar proses penyimpanan dan distribusi cairan dapat berjalan dengan aman, efisien, dan terkendali.

Pada proses industri, level cairan yang terlalu rendah dapat menyebabkan pompa bekerja tanpa fluida (dry running) sehingga mengakibatkan kerusakan peralatan. Sebaliknya, level cairan yang terlalu tinggi dapat menyebabkan overflow yang berpotensi menimbulkan kerugian produksi maupun bahaya keselamatan kerja. Oleh karena itu, diperlukan suatu sistem monitoring yang mampu memberikan informasi kondisi tangki secara real-time serta mampu melakukan pengendalian sederhana terhadap aktuator seperti valve dan pompa berdasarkan kondisi level cairan.

Kami memilih bahasa pemrograman Rust dalam project ini karena memiliki performa tinggi, keamanan memori yang baik, serta kemampuan pengolahan sistem yang stabil untuk pengembangan aplikasi monitoring industri. Rust juga mendukung paradigma pemrograman modern seperti Object Oriented Programming (OOP) dan modular programming sehingga mempermudah pengembangan sistem yang terstruktur dan mudah dipelihara.

Project ini mengimplementasikan sistem monitoring level tangki berbasis console menggunakan bahasa Rust dengan konsep pemrograman berbasis objek melalui struktur (struct) dan implementasi fungsi (impl). Sistem dapat menerima input parameter tangki, membaca nilai sensor level, menghitung volume cairan, menentukan status kondisi tangki berdasarkan batas Low Level (LL) dan High High (HH), serta mengontrol status inlet valve, outlet valve, dan pompa secara otomatis.

Melalui project ini diharapkan dapat memberikan pemahaman mengenai penerapan dasar sistem otomasi industri menggunakan simulasi perangkat lunak serta menjadi media pembelajaran mengenai integrasi logika kontrol, monitoring proses, dan pemrograman Rust dalam bidang instrumentasi dan kontrol industri.

2 Studi Kasus Instrumentasi Sistem Monitoring Level Tangki

Pada sebuah industri pengolahan air digunakan sebuah tangki penyimpanan air proses untuk mendukung kebutuhan produksi. Tangki tersebut memiliki kapasitas maksimum sebesar 10.000 liter dengan tinggi total 5 meter. Untuk menjaga kestabilan proses dan mencegah terjadinya overflow maupun kekosongan tangki, digunakan sistem monitoring level berbasis sensor level.

Sistem monitoring bekerja dengan membaca tinggi cairan di dalam tangki menggunakan sensor level dalam satuan meter. Data pembacaan sensor kemudian diproses oleh program untuk menentukan volume cairan, persentase isi tangki, serta kondisi operasional tangki berdasarkan batas Low Level (LL) dan High High (HH).

Parameter instrumentasi yang digunakan pada studi kasus ini ditunjukkan pada Tabel berikut.

Tabel 1: Parameter Instrumentasi Tangki

Parameter	Nilai
Kapasitas Maksimum Tangki	10.000 Liter
Tinggi Tangki	5 Meter
Low Level (LL)	1 Meter
High High (HH)	4.5 Meter
Pembacaan Sensor	3 Meter

Dalam sistem ini, sensor level membaca tinggi cairan sebesar 3 meter. Karena operator industri lebih mudah memahami kapasitas fluida dalam satuan volume, maka dilakukan konversi level cairan dari meter menjadi liter menggunakan persamaan berikut:

$$\text{Volume} = \left(\frac{\text{Level}}{\text{Tinggi Tangki}} \right) \times \text{Kapasitas Maksimum} \quad (1)$$

Substitusi nilai:

$$\text{Volume} = \left(\frac{3}{5} \right) \times 10000 \quad (2)$$

Sehingga diperoleh hasil:

$$\text{Volume} = 6000 \text{ Liter} \quad (3)$$

Artinya, ketika sensor membaca level cairan setinggi 3 meter, volume air di dalam tangki adalah sebesar 6000 liter.

Selain volume, sistem juga menghitung persentase isi tangki menggunakan persamaan:

$$\text{Persentase} = \left(\frac{\text{Level}}{\text{Tinggi Tangki}} \right) \times 100\% \quad (4)$$

Substitusi nilai:

$$\text{Persentase} = \left(\frac{3}{5} \right) \times 100\% \quad (5)$$

Maka diperoleh:

$$\text{Persentase} = 60\% \quad (6)$$

Berdasarkan logika kontrol instrumentasi, sistem menggunakan dua batas alarm utama yaitu Low Level (LL) dan High High (HH). Jika level cairan berada di bawah LL maka inlet valve akan terbuka dan pompa akan aktif untuk mengisi tangki. Sebaliknya, jika level cairan mencapai HH maka outlet valve akan terbuka untuk mencegah overflow.

Pada studi kasus ini, level sensor berada pada kondisi:

$$1 < 3 < 4.5 \quad (7)$$

Karena nilai sensor masih berada di antara batas LL dan HH, maka kondisi tangki dinyatakan NORMAL. Status aktuator sistem adalah:

- Inlet Valve : CLOSE
- Pump : OFF
- Outlet Valve : CLOSE

Melalui studi kasus ini dapat dipahami bahwa sistem monitoring level tangki mampu melakukan pembacaan level cairan, konversi satuan meter ke liter, perhitungan persentase isi tangki, serta simulasi logika kontrol instrumentasi secara otomatis menggunakan bahasa pemrograman Rust.

3 Algoritma Program

Algoritma pada sistem monitoring level tangki dibuat untuk melakukan pemantauan level cairan di dalam tangki berdasarkan parameter yang dimasukkan oleh pengguna. Program ini dikembangkan menggunakan bahasa pemrograman Rust dengan pendekatan *Object Oriented Programming* (OOP) menggunakan *struct* dan *method*. Sistem bekerja dengan membaca nilai sensor level, kemudian menentukan kondisi tangki berdasarkan batas *Low Low* (LL) dan *High High* (HH). Selain itu, sistem juga mengontrol status inlet valve, outlet valve, dan pompa secara otomatis sesuai kondisi level cairan.

Tahap awal program dimulai dengan menampilkan judul aplikasi "Sistem Monitoring Level Tangki" kemudian program masuk ke menu utama. Pada menu utama terdapat tiga pilihan yaitu input parameter dan sensor, melihat dashboard, dan keluar dari program. Pengguna diminta memasukkan pilihan menu sesuai kebutuhan sistem.

Jika pengguna memilih menu input data, maka program akan meminta beberapa parameter penting yaitu kapasitas tangki dalam liter, tinggi tangki dalam meter, batas *Low Low* (LL), batas *High High* (HH), dan pembacaan sensor level saat ini. Seluruh data yang dimasukkan akan divalidasi untuk memastikan bahwa nilai yang diberikan berupa angka dan tidak bernilai negatif. Apabila data tidak valid maka program akan menampilkan pesan kesalahan dan meminta pengguna menginput ulang data yang benar.

Setelah data valid, program akan membuat objek `TankMonitoring` berdasarkan parameter yang telah dimasukkan. Selanjutnya dilakukan proses perhitungan volume cairan di dalam tangki menggunakan persamaan:

$$\text{Volume} = \left(\frac{\text{Reading Sensor}}{\text{Tinggi}} \right) \times \text{Kapasitas}$$

Selain itu, program juga menghitung persentase level cairan menggunakan persamaan:

$$\text{Persentase} = \left(\frac{\text{Reading Sensor}}{\text{Tinggi}} \right) \times 100\%$$

Hasil pembacaan sensor kemudian dibandingkan dengan batas LL dan HH untuk menentukan kondisi tangki. Jika nilai sensor lebih kecil atau sama dengan LL maka kondisi tangki dinyatakan *Low Low* dan alarm aktif. Pada kondisi ini inlet valve dibuka, pompa dinyalakan, dan outlet valve ditutup untuk mengisi tangki. Jika nilai sensor berada di antara LL dan HH maka kondisi tangki dianggap normal sehingga inlet valve, pompa, dan outlet valve berada pada kondisi tertutup atau tidak aktif. Sedangkan apabila nilai sensor lebih besar atau sama dengan HH maka kondisi tangki dinyatakan *High High*. Pada kondisi ini alarm aktif, inlet valve ditutup, pompa dimatikan, dan outlet valve dibuka untuk mengurangi level cairan di dalam tangki.

Setelah seluruh proses selesai dilakukan, program menampilkan dashboard monitoring yang berisi parameter tangki, level sensor, volume cairan, persentase level, kondisi tangki, serta status inlet valve, pompa, dan outlet valve. Dashboard ini memudahkan pengguna untuk memonitor kondisi tangki secara real-time melalui tampilan terminal. Setelah dashboard ditampilkan, program kembali ke menu utama sehingga pengguna dapat melakukan input data kembali atau keluar dari aplikasi.

3.1 Flowchart Sistem

Flowchart sistem monitoring level tangki digunakan untuk menggambarkan alur kerja program secara visual mulai dari program dijalankan hingga program selesai. Flowchart ini membantu dalam memahami urutan proses, pengambilan keputusan, dan hubungan antar bagian program secara sistematis.

Pada tahap awal, flowchart dimulai dengan proses *Start* kemudian program menampilkan judul aplikasi sistem monitoring level tangki. Setelah itu program menampilkan menu utama yang terdiri dari tiga pilihan yaitu input parameter dan sensor, melihat dashboard, dan keluar dari program. Pengguna kemudian memasukkan pilihan menu sesuai kebutuhan.

Jika pengguna memilih menu pertama, maka program akan masuk ke proses input data tangki dan sensor. Selanjutnya dilakukan validasi input untuk memastikan bahwa data berupa angka valid dan tidak bernilai negatif. Jika data tidak valid maka sistem menampilkan pesan kesalahan dan pengguna diminta melakukan input ulang.

Apabila data valid maka program membuat objek monitoring tangki dan melanjutkan proses perhitungan volume serta persentase level cairan. Setelah proses perhitungan selesai, sistem melakukan pengecekan kondisi tangki berdasarkan nilai sensor terhadap batas LL dan HH.

Jika sensor berada pada kondisi LL maka sistem memberikan status *Low Low* dengan alarm aktif, inlet valve terbuka, pompa aktif, dan outlet valve tertutup. Jika sensor berada pada kondisi normal maka seluruh aktuator berada dalam kondisi normal. Sedangkan jika sensor mencapai HH maka sistem memberikan status *High High* dengan alarm aktif, inlet valve tertutup, pompa mati, dan outlet valve terbuka.

Setelah kondisi ditentukan, sistem menampilkan dashboard monitoring yang berisi seluruh informasi parameter dan status sistem. Program kemudian kembali ke menu utama sehingga pengguna dapat melakukan monitoring kembali atau keluar dari program.

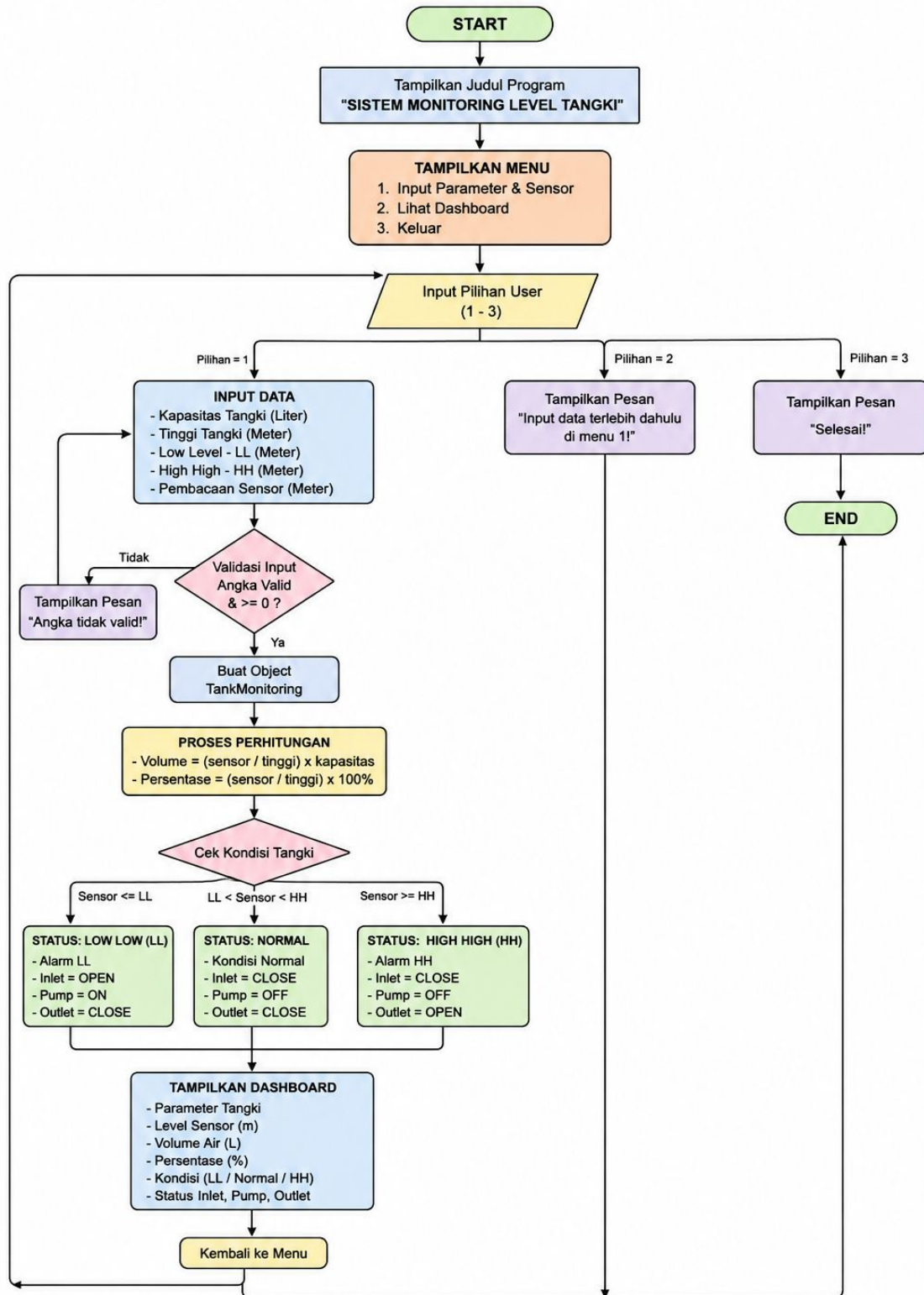
Jika pengguna memilih menu kedua tanpa melakukan input data terlebih dahulu, maka sistem menampilkan pesan bahwa pengguna harus menginput data terlebih dahulu pada menu pertama. Sedangkan jika pengguna memilih menu ketiga maka sistem menampilkan pesan selesai dan program berakhir pada proses *End*.

3.2 Penjelasan Flowchart

Berikut merupakan penjelasan dari simbol dan proses yang terdapat pada flowchart sistem monitoring level tangki:

1. **Start**
Menunjukkan awal program dijalankan.
2. **Tampilkan Judul Program**
Program menampilkan judul sistem monitoring level tangki pada terminal.
3. **Tampilkan Menu**
Sistem menampilkan menu utama yang berisi pilihan input data, dashboard, dan keluar.
4. **Input Pilihan User**
Pengguna memasukkan pilihan menu dari angka 1 sampai 3.
5. **Input Data Tangki**
Pengguna memasukkan parameter kapasitas tangki, tinggi tangki, batas LL, batas HH, dan nilai sensor.
6. **Validasi Input**
Sistem memeriksa apakah input berupa angka valid dan tidak bernilai negatif.
7. **Proses Perhitungan**
Sistem menghitung volume cairan dan persentase level tangki berdasarkan pembacaan sensor.
8. **Cek Kondisi Tangki**
Sistem membandingkan nilai sensor terhadap batas LL dan HH untuk menentukan kondisi tangki.
9. **Tampilkan Dashboard**
Sistem menampilkan hasil monitoring berupa level cairan, volume, persentase, kondisi tangki, dan status aktuator.
10. **End**
Menunjukkan akhir dari program.

DIAGRAM ALGORITMA PROGRAM SISTEM MONITORING LEVEL TANGKI



Gambar 1: Flowchart Sistem Monitoring Level Tangki

4 KONSEP PEMROGRAMAN BERBASIS OBJEK (OOP)

4.1 Pengertian Object Oriented Programming (OOP)

Object Oriented Programming (OOP) atau pemrograman berbasis objek merupakan paradigma pemrograman yang menggunakan objek sebagai dasar utama dalam pengembangan program. Objek digunakan untuk merepresentasikan data serta fungsi yang saling berhubungan di dalam suatu sistem. Konsep OOP bertujuan untuk mempermudah pengelolaan program agar lebih terstruktur, mudah dikembangkan, mudah dipelihara, dan dapat digunakan kembali.

Pada project sistem monitoring level tangki ini, konsep OOP diterapkan menggunakan bahasa pemrograman Rust dengan memanfaatkan *struct* dan *implementation block* (`impl`). Struktur program dibuat agar data tangki dan seluruh proses monitoring berada di dalam satu objek sehingga sistem menjadi lebih modular dan efisien.

4.2 Penerapan OOP pada Sistem Monitoring Level Tangki

Pada program sistem monitoring level tangki, konsep OOP digunakan untuk mengelompokkan seluruh parameter dan fungsi monitoring ke dalam objek `TankMonitoring`. Objek ini menyimpan data kapasitas tangki, tinggi tangki, batas *Low Low* (LL), batas *High High* (HH), dan pembacaan sensor level.

Berikut merupakan deklarasi *struct* yang digunakan pada program:

```
struct TankMonitoring {  
    max_capacity: f32,  
    height: f32,  
    ll: f32,  
    hh: f32,  
    sensor_reading: f32,  
}
```

Struct tersebut berfungsi sebagai blueprint atau rancangan objek yang menyimpan seluruh data monitoring tangki. Setiap variabel memiliki tipe data `f32` karena digunakan untuk menyimpan nilai desimal.

4.3 Konsep Encapsulation

Encapsulation merupakan konsep pembungkusan data dan method ke dalam satu objek. Pada program ini seluruh parameter tangki dan fungsi monitoring dibungkus di dalam struct `TankMonitoring`. Hal ini membuat data dan proses monitoring menjadi terorganisir dengan baik.

Method yang terdapat pada objek antara lain:

- `new()` untuk membuat objek baru.

- `meter_to_liter()` untuk mengubah level meter menjadi volume liter.
- `get_inlet_status()` untuk menentukan status inlet valve dan pompa.
- `get_outlet_status()` untuk menentukan status outlet valve.
- `get_condition()` untuk menentukan kondisi tangki.
- `display()` untuk menampilkan dashboard monitoring.

Seluruh method tersebut berada di dalam blok `impl TankMonitoring`.

4.4 Konsep Constructor

Constructor merupakan fungsi khusus yang digunakan untuk membuat objek baru. Pada Rust constructor dibuat menggunakan method `new()`.

Berikut constructor yang digunakan pada project:

```
fn new(
    max_capacity: f32,
    height: f32,
    ll: f32,
    hh: f32,
    sensor_reading: f32
) -> Self {
    TankMonitoring {
        max_capacity,
        height,
        ll,
        hh,
        sensor_reading,
    }
}
```

Constructor tersebut digunakan untuk menginisialisasi seluruh data tangki ketika objek dibuat.

4.5 Keuntungan Penggunaan OOP pada Project

Penerapan konsep OOP pada sistem monitoring level tangki memberikan beberapa keuntungan, yaitu:

1. Program menjadi lebih terstruktur dan rapi.
2. Data dan fungsi monitoring tersimpan dalam satu objek sehingga mudah dikelola.
3. Program lebih mudah dikembangkan apabila ingin menambahkan fitur baru seperti alarm otomatis atau integrasi sensor IoT.

4. Method dapat digunakan kembali tanpa perlu menulis kode berulang.
5. Mempermudah proses debugging dan maintenance program.

Dengan penerapan konsep OOP, sistem monitoring level tangki menjadi lebih modular, efisien, dan mudah dipahami dalam proses pengembangan perangkat lunak instrumentasi industri.

5 Implementasi Komputasi Numerik

Komputasi numerik digunakan untuk mengolah data mentah sensor:

- **Koreksi Linear:** Menggunakan penambahan bias secara konstan untuk mendapatkan nilai riil.
- **Konversi Skala:** Melakukan pemetaan (*mapping*) nilai linear dari domain meter ($0 - h$) ke domain volume ($0 - V_{max}$).

6 Penjelasan Program Rust

6.1 Latar Belakang

Program ini merupakan simulasi sistem monitoring level tangki industri yang dibuat menggunakan bahasa pemrograman Rust. Sistem ini dirancang untuk memantau kapasitas air, memberikan peringatan visual saat kondisi kritis (*Low Low* atau *High High*), serta mengontrol status katup (*valve*) dan pompa secara otomatis berdasarkan pembacaan sensor.

6.2 Kode Sumber (Source Code)

Berikut adalah implementasi lengkap program dalam Rust:

```
1 use std::io;
2 use std::io::Write;
3
4 // =====
5 // STRUCT - Pemrograman Berbasis Objek
6 // =====
7
8 struct TankMonitoring {
9     max_capacity: f32,      // Kapasitas maksimal (Liter)
10    height: f32,             // Tinggi tangki (Meter)
11    ll: f32,                 // Low Level (Meter)
12    hh: f32,                 // High High (Meter)
13    sensor_reading: f32,     // Pembacaan sensor (Meter)
14 }
15
16 impl TankMonitoring {
17     // Konstruktor
18     fn new(max_capacity: f32, height: f32, ll: f32, hh: f32,
19 sensor_reading: f32) -> Self {
20         TankMonitoring {
21             max_capacity,
22             height,
23             ll,
24             hh,
25             sensor_reading,
26         }
27
28         // Konversi Meter ke Liter
29         fn meter_to_liter(&self, meter: f32) -> f32 {
30             (meter / self.height) * self.max_capacity
31         }
32
33         // Status Inlet Valve & Pump
34         fn get_inlet_status(&self) -> (String, String) {
```

```

35         if self.sensor_reading <= self.ll {
36             ("OPEN".to_string(), "ON".to_string())
37         } else {
38             ("CLOSE".to_string(), "OFF".to_string())
39         }
40     }
41
42     // Status Outlet Valve
43     fn get_outlet_status(&self) -> String {
44         if self.sensor_reading >= self.hh {
45             "OPEN".to_string()
46         } else {
47             "CLOSE".to_string()
48         }
49     }
50
51     // Status Kondisi Tangki
52     fn get_condition(&self) -> String {
53         if self.sensor_reading >= self.hh {
54             "HIGH HIGH (HH) - ALARM!".to_string()
55         } else if self.sensor_reading <= self.ll {
56             "LOW LOW (LL) - ALARM!".to_string()
57         } else {
58             "NORMAL".to_string()
59         }
60     }
61
62     // Tampilkan Dashboard
63     fn display(&self) {
64         println!("\n{}", "=" .repeat(60));
65         println!("{}", "DASHBOARD TANGKI");
66         println!("{}", "=" .repeat(60));
67
68         println!("\n    PARAMETER:");
69         println!("    Kapasitas: {:.2} L | Tinggi: {:.2} m",
self.max_capacity, self.height);
70         println!("    LL: {:.2} m | HH: {:.2} m", self.ll, self
.hh);
71
72         let volume = self.meter_to_liter(self.sensor_reading)
;
73         let percentage = (self.sensor_reading / self.height)
* 100.0;
74
75         println!("\n    DATA SAAT INI:");
76         println!("    Level Sensor: {:.2} m", self.
sensor_reading);
77         println!("    Volume Air: {:.2} L", volume);
78         println!("    Persentase: {:.1}%", percentage);

```

```

79
80     println!("\ n      STATUS SISTEM:");
81     println!("  Kondisi: {}", self.get_condition());
82     let (inlet, pump) = self.get_inlet_status();
83     let outlet = self.get_outlet_status();
84     println!("  INLET: {} | PUMP: {} | OUTLET: {}", inlet
, pump, outlet);
85
86     println!("\n{}", "=" .repeat(60));
87 }
88 }
89
90 // =====
91 // FUNGSI HELPER
92 // =====
93
94 fn input_f32(prompt: &str) -> f32 {
95     loop {
96         print!("{}", prompt);
97         let _ = Write::flush(&mut io::stdout());
98
99         let mut input = String::new();
100         io::stdin().read_line(&mut input).unwrap();
101
102         match input.trim().parse::<f32>() {
103             Ok(value) if value >= 0.0 => return value,
104             _ => println!("      Angka tidak valid!\n"),
105         }
106     }
107 }
108
109 // =====
110 // MAIN PROGRAM
111 // =====
112
113 fn main() {
114     println!("\n
+=====+
");
115     println!("|      SISTEM MONITORING LEVEL TANGKI - VERSI
LENGKAP      |");
116     println!("
+=====+\n
n");
117
118     loop {
119         println!("\ n      MENU:");
120         println!("1. Input Parameter & Sensor");
121         println!("2. Lihat Dashboard");

```

```

122     println!("3. Keluar");
123     print!("Pilih (1-3): ");
124     let _ = Write::flush(&mut io::stdout());
125
126     let mut choice = String::new();
127     io::stdin().read_line(&mut choice).unwrap();
128
129     match choice.trim() {
130         "1" => {
131             let max_cap = input_f32("Kapasitas tangki (
132 Liter): ");
133             let height = input_f32("Tinggi tangki (Meter)
134 : ");
135             let ll = input_f32("Batas Low Level - LL (
136 Meter): ");
137             let hh = input_f32("Batas High High - HH (
138 Meter): ");
139             let sensor = input_f32("Pembacaan sensor saat
140 ini (Meter): ");
141
142             let tank = TankMonitoring::new(max_cap,
143 height, ll, hh, sensor);
144             tank.display();
145         }
146         "2" => println!("\n      Input data terlebih
147 dahulu di menu 1!"),
148         "3" => {
149             println!("\n      Selesai!\n");
150             break;
151         }
152         _ => println!("      Pilihan tidak valid!\n"),
153     }
154 }

```

Listing 1: Implementasi Lengkap Sistem Monitoring Tangki dalam Rust

6.3 Penjelasan Struktur Program

6.3.1 Struct TankMonitoring

Struktur data ini merepresentasikan tangki fisik. Di dalamnya terdapat lima field utama:

- `max_capacity`: Volume maksimum tangki (Liter).
- `height`: Ketinggian fisik tangki (Meter).
- `ll` (*Low Low*): Ambang batas bawah untuk memicu pompa.
- `hh` (*High High*): Ambang batas atas untuk memicu pembuangan.

- **sensor_reading**: Data aktual dari sensor level.

6.3.2 Implementasi Method (impl)

Bagian ini berisi logika bisnis atau fungsi yang melekat pada objek tangki:

1. **new()**: Berperan sebagai *constructor* untuk membuat instansi baru dari tangki. Method ini menginisialisasi seluruh field dengan nilai yang diberikan pengguna.
2. **meter_to_liter()**: Menggunakan perbandingan linier:

$$\text{Volume} = \left(\frac{\text{sensor}}{\text{height}} \right) \times \text{max_capacity}$$

untuk menghitung volume berdasarkan pembacaan meter.

3. **get_inlet_status()**: Logika kontrol yang mengembalikan tuple berisi status inlet valve dan pompa. Jika air $\leq$ LL, maka Inlet Valve dibuka (*OPEN*) dan Pompa dinyalakan (*ON*). Selain itu kedua aktuator akan tertutup/mati.
4. **get_outlet_status()**: Menentukan status outlet valve. Jika air $\geq$ HH, maka outlet valve dibuka (*OPEN*) untuk mencegah *overflow*. Dalam kondisi lain, outlet tertutup.
5. **get_condition()**: Mengembalikan string yang mendeskripsikan kondisi tangki. Terdapat tiga status: HIGH HIGH ALARM, LOW LOW ALARM, atau NORMAL.
6. **display()**: Menampilkan dashboard monitoring lengkap ke terminal dengan format yang terstruktur dan mudah dibaca.

6.3.3 Fungsi Helper input_f32

Fungsi ini menangani validasi input dari pengguna. Menggunakan *loop* dan *pattern matching* (**match**) untuk memastikan pengguna memasukkan angka desimal positif. Jika pengguna memasukkan teks atau angka negatif, program akan menampilkan pesan kesalahan dan meminta input ulang tanpa mengalami *crash*.

6.4 Logika Operasional Sistem

Sistem bekerja dengan prinsip *State Monitoring*. Berikut adalah ringkasan logika keputusannya:

Logika ini memastikan bahwa:

- Ketika level terlalu rendah (LL), sistem secara otomatis mengaktifkan pompa dan membuka inlet untuk mengisi tangki.
- Ketika level normal, semua aktuator berada dalam keadaan tidak aktif.
- Ketika level terlalu tinggi (HH), sistem membuka outlet valve untuk mengeluarkan kelebihan air dan mencegah overflow.

Tabel 2: Logika Kontrol Sistem Monitoring Tangki

Kondisi Sensor	Status	Inlet/Pump	Outlet
Sensor $\leq$ LL	LOW LOW ALARM	OPEN / ON	CLOSE
LL $\leq$ Sensor $\leq$ HH	NORMAL	CLOSE / OFF	CLOSE
Sensor $\geq$ HH	HIGH HIGH ALARM	CLOSE / OFF	OPEN

7 Hasil Pengujian

Sistem telah diuji dengan skenario berikut:

- **Konfigurasi Awal:** Kapasitas 10.000 L, Tinggi 5 m, Batas LL 1 m, HH 4.5 m.
- **Skenario 1 - Kondisi Normal:** Sensor membaca 3 m. Sistem menghitung volume 6.000 L (60)
- **Skenario 2 - Kondisi Low Low:** Sensor membaca 0.8 m. Sistem menghitung volume 1.600 L (16)
- **Skenario 3 - Kondisi High High:** Sensor membaca 4.8 m. Sistem menghitung volume 9.600 L (96)

Hasil pengujian menunjukkan bahwa semua kondisi logika bekerja sesuai spesifikasi yang dirancang.

8 Analisis Hasil

Implementasi sistem monitoring dalam Rust menunjukkan beberapa keunggulan signifikan:

1. **Type Safety:** Rust memberikan jaminan tipe data yang ketat, sehingga mengurangi risiko error pada saat runtime. Penggunaan `f32` untuk semua parameter numerik memastikan konsistensi presisi.
2. **Memory Safety:** Rust mengelola memori secara otomatis tanpa garbage collector, sehingga program berjalan dengan overhead minimal.
3. **Input Validation:** Fungsi `input_f32` menggunakan error handling yang robust dengan loop untuk memastikan hanya nilai valid yang diterima.
4. **Code Readability:** Penggunaan *struct* dan *impl* membuat kode menjadi modular dan mudah dipahami. Setiap method memiliki tanggung jawab tunggal (Single Responsibility Principle).
5. **Conversion Accuracy:** Konversi meter ke liter menggunakan formula linier yang akurat dan konsisten dengan perhitungan manual yang dilakukan pada studi kasus.

9 Kesimpulan

Pengembangan sistem monitoring level tangki berbasis Rust ini membuktikan bahwa:

1. Konsep OOP sangat efektif untuk memodelkan aset fisik seperti tangki industri.
2. Logika *safety interlocking* yang diterapkan berhasil menjamin bahwa aktuator (inlet/outlet/pompa) akan merespons sesuai dengan kondisi level yang telah dianalisis.
3. Rust sebagai bahasa pemrograman sistem modern mampu menghadirkan keseimbangan sempurna antara performa, keamanan, dan aksesibilitas untuk pengembangan aplikasi instrumentasi industri.
4. Program yang telah dikembangkan dapat dengan mudah diperluas untuk integrasi dengan hardware sensor IoT, database logging, atau antarmuka web tanpa memerlukan restrukturisasi besar.